

TASK - 1.1.A

1.

 - {EmpID}
 - {EmpID, Name}
 - {EmpID, SSN}
 - {EmpID, Email}
 - {EmpID, Phone, Department}
 - {EmpID, Name, Department, Salary}
2.

 - {EmpID}
 - {SSN}
 - {Email}
 - {Phone}

3.

 - {EmpID}

It is the only truly unique identifier for a real database.

SSN, Email, Phone could also be primary keys for this particular table.

4.

No, based strictly on the data shown, two employees cannot have the same phone number.

Phone
555-0101
555-0102
555-0103

All three phone numbers in the provided table are completely distinct. However, if in the database the Phone column does not have the UNIQUE constraint. then there can be two same numbers.

TASK - 1.1,B

1. {StudentID, CourseCode, Section, Semester, Year}
2.
 - StudentID: Identifies which student is registering.
 - CourseCode: Identifies which course the student is taking.
 - Section: Needed because a course can have multiple sections (e.g., Section 01, Section 02). The business rule states a student cannot register for the same course section in the same semester, implying they could register for a different section.
 - Semester: Needed because the rule states a student can take the same course in different semesters.
 - Year: Needed in conjunction with Semester because "Fall 2023" is a different semester than "Fall 2024."
3. There are no additional candidate keys.

TASK - 1.2

1.
 - Student.AdvisorID references Professor.ProfID
 - Course.DepartmentCode references Department.DeptCode
 - Professor.Department references Department.DeptCode
 - Department.ChairID references Professor.ProfID
 - Enrollment.CourseID references Course.CourseID
 - Enrollment.StudentID references Student.StudentID

TASK - 2.1

1 Strong Entities:

- Patient: Exists independently. Has its own unique identifier (PatientID).
- Doctor: Exists independently. Has its own unique identifier (DoctorID).
- Department: Exists independently. Has its own unique identifier (DeptCode).
- Appointment: Exists as a specific event linking a patient and a doctor. (It has a surrogate key, AppointmentID, making it strong, though it is functionally dependent on Patient and Doctor).
- Prescription: Exists as a specific medical order linking a patient, a doctor, and medication.

Weak Entity:

- Hospital Room: The requirement states "room 101 in Cardiology is different from room 101 in Neurology." A room cannot exist without a department, and its unique identifier depends on the department it belongs to. Therefore, it is a weak entity.

2 Patient

- PatientID (Simple, Primary Key)
- Name (Composite: FirstName, LastName)
- Birthdate (Simple)
- Address (Composite: Street, City, State, Zip)
- Phone (Multi-valued)
- InsuranceInfo (Composite: Provider, PolicyNumber)

Doctor

- DoctorID (Simple, Primary Key)
- Name (Composite: FirstName, LastName)
- Specialization (Multi-valued)
- Phone (Multi-valued)
- OfficeLocation (Simple)

TASK - 2.1

2 Department

- DeptCode (Simple, Primary Key)
- Name (Simple)
- Location (Simple)

Appointment

- AppointmentID (Simple, Primary Key - surrogated added for uniqueness)
- DateTime (Composite: Date, Time)
- Purpose (Simple)
- Notes (Simple)

Prescription

- PrescriptionID (Simple, Primary Key)
- Medication (Simple)
- Dosage (Simple)
- Instructions (Simple)

Hospital Room (Weak Entity)

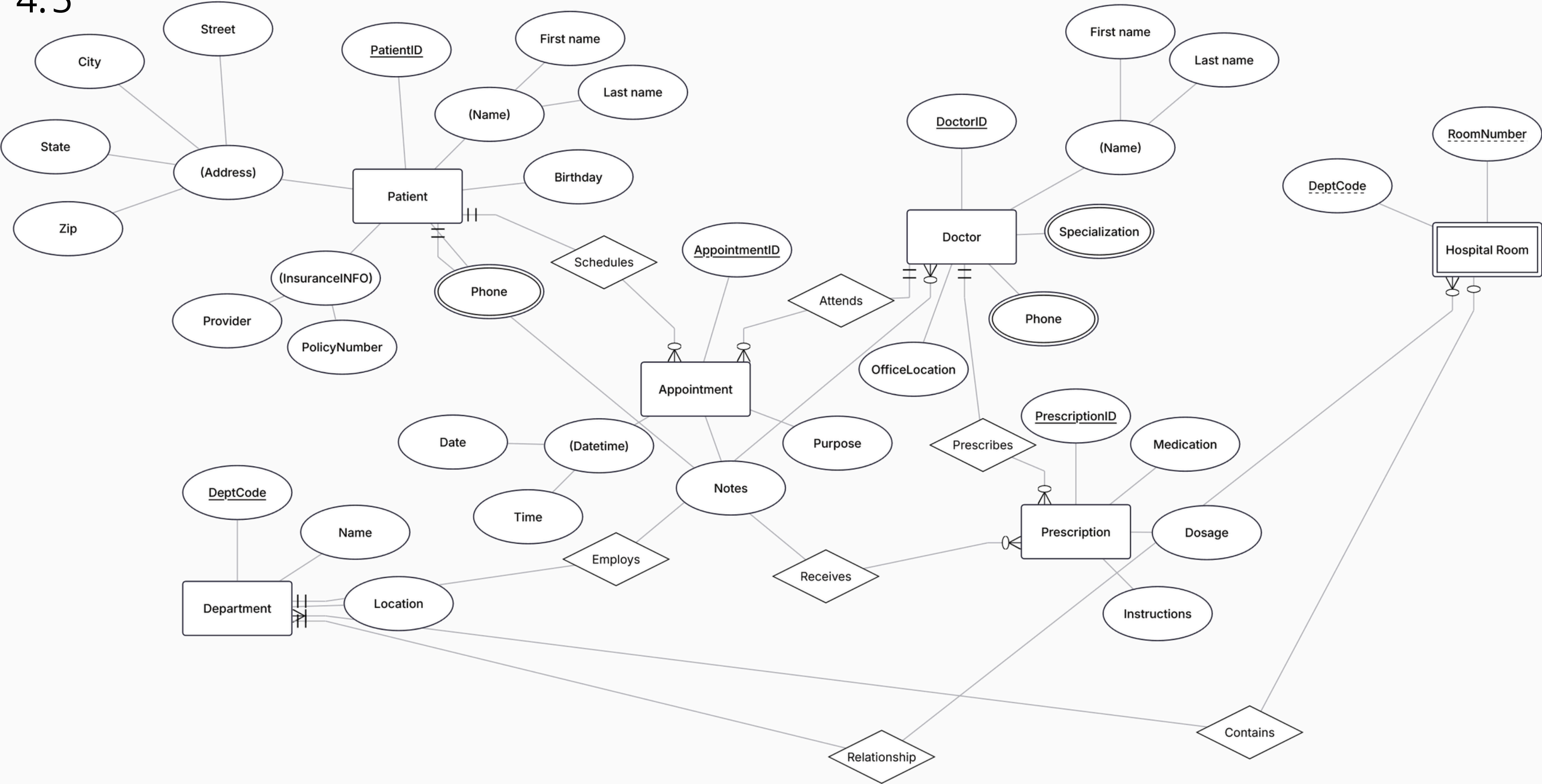
- RoomNumber (Simple, Partial Key)
- DeptCode is the identifying attribute inherited from Department

3

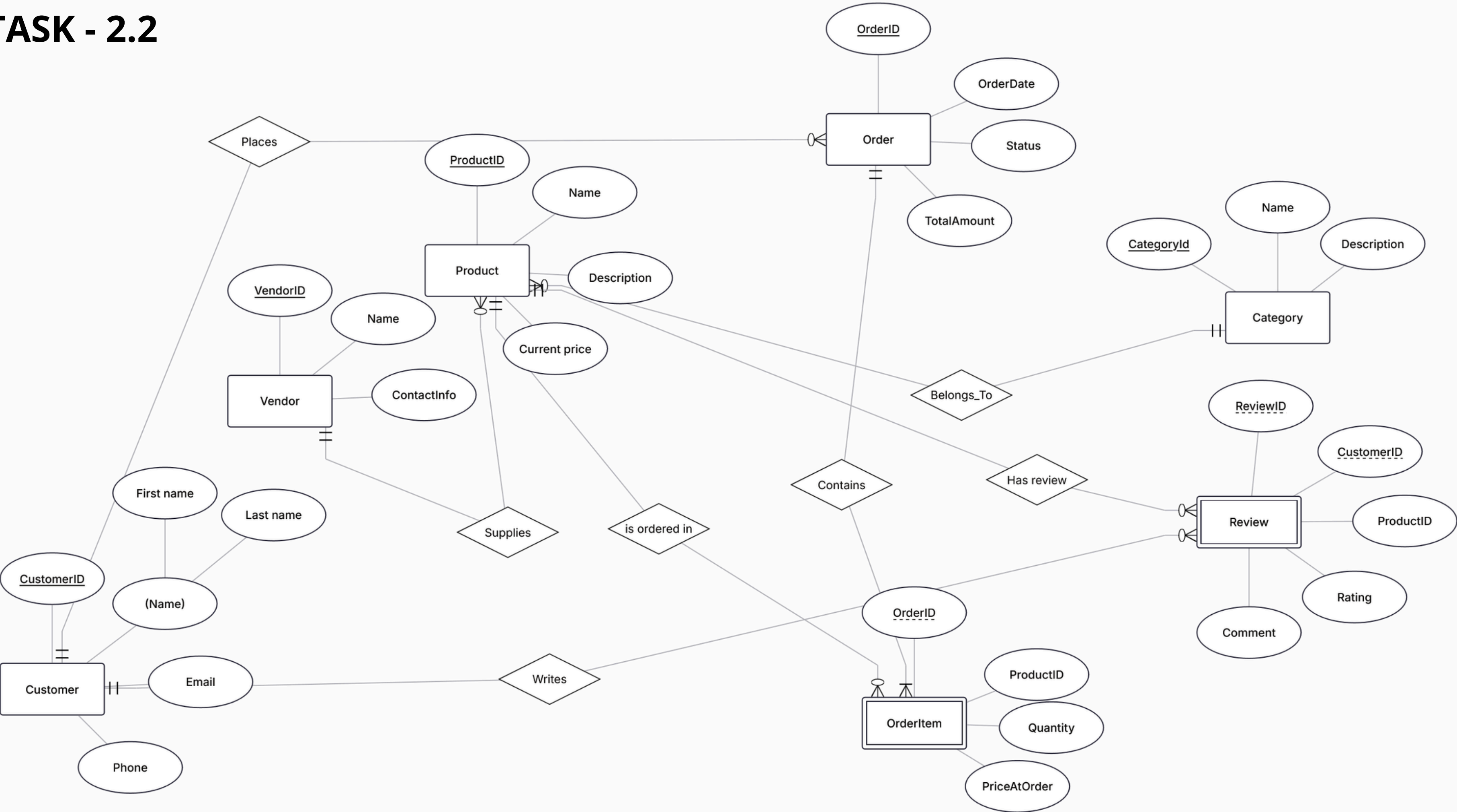
1. Patient - Appointment: 1:N
2. Doctor - Appointment: 1:N
3. Patient - Prescription: 1:N
4. Doctor - Prescription: 1:N
5. Department - Doctor: 1:N
6. Department - Hospital Room: 1:N (Identifying Relationship)

TASK - 2.1

4.5



TASK - 2.2



TASK - 2.2

- 2.** The weak entity is OrderItem. It is weak because it has no standalone existence; an order item only exists as part of a specific Order. If the order is deleted, the item is deleted. Furthermore, it lacks a unique identifier of its own, relying on a composite key of {OrderID, ProductID} to be uniquely identified.

TASK - 2.2

3. M:N Relationship: Order and Product

An Order can contain many Products. A Product can be in many Orders. If we simply drew a diamond between Order and Product, we would have nowhere to store the specific details of that individual purchase.

We resolve this M:N relationship by creating the associative entity OrderItem. This entity captures the attributes that belong to the relationship, not the entities themselves:

Quantity: How many of this specific product were bought in this specific order.

PriceAtOrder: The price of the product at the exact time the order was placed

TASK - 4.1

1.

- StudentID → StudentName, StudentMajor
- ProjectID → ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate
- SupervisorID → SupervisorName, SupervisorDept
- (StudentID, ProjectID, Role) → HoursWorked

2.

Redundancy: Student information repeated on every project they join in.

Update: If you need to update a project's name, you have to update all the rows with students in that project.

Insert: You cannot add a new project until a student is assigned to it (you need a studentID)

Delete: Deleting the only row with the project deletes the project entirely

3.

Already in 1NF. No violations.

TASK - 4.1

4.

Primary keys: StudentID, ProjectID.

Partial dependencies:

StudentName and StudentMajor depend only on StudentID.

ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate depend on only on ProjectID.

Student: StudentID, StudentName, StudentMajor.

PK: StudentID.

Project: ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, StartDate, EndDate.

PK: ProjectID.

StudentProjectAssignment: StudentID, ProjectID, Role, HoursWorked

PK: StudentID, ProjectID, Role.

FK: StudentID references Student, ProjectID references Project.

TASK - 4.1

5.

Transitive dependencies:

ProjectID $\rightarrow$ SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

Student: StudentID, StudentName, StudentMajor.

PK: StudentID.

Project: ProjectID, ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate.

PK: ProjectID.

FK: SupervisorID.

Supervisor: SupervisorID, SupervisorName, SupervisorDept.

PK: SupervisorID.

StudentProjectAssignment: StudentID, ProjectID, Role, HoursWorked

PK: StudentID, ProjectID, Role.

FK: StudentID references Student, ProjectID references Project.

TASK - 4.2

1. {StudentID, CourseID, TimeSlot, Room}
2.
 - StudentID $\rightarrow$ StudentMajor (Rule: Each student has exactly one major)
 - CourseID $\rightarrow$ CourseName (Rule: Each course has a fixed name)
 - InstructorID $\rightarrow$ InstructorName (Rule: Each instructor has exactly one name)
 - Room $\rightarrow$ Building (Rule: Rooms are unique across campus, so the room determines the building)
 - CourseID, TimeSlot, Room $\rightarrow$ InstructorID (Rule: Each course section is taught by one instructor at one time in one room)
3. No, the table is NOT in BCNF.

TASK - 4.2

4. Student(StudentID, StudentMajor)

PK: StudentID

Course(CourseID, CourseName)

PK: CourseID

Instructor(InstructorID, InstructorName)

PK: InstructorID

RoomInfo(Room, Building)

PK: Room

CourseSection(CourseID, TimeSlot, InstructorID, Room)

PK: CourseID, TimeSlot, Room

FK: InstructorID references Instructor.InstructorID

CourseID references Course.CourseID

Room references RoomInfo(Room)

TASK - 4.2

4. Enrollment (StudentID, CourseID, TimeSlot, Room)

PK: {StudentID, CourseID, TimeSlot, Room}

FK: StudentID references Student(StudentID)

FK: {CourseID, TimeSlot, Room} references Section(CourseID, TimeSlot, Room)

5. • Lossless Join (Data Loss):

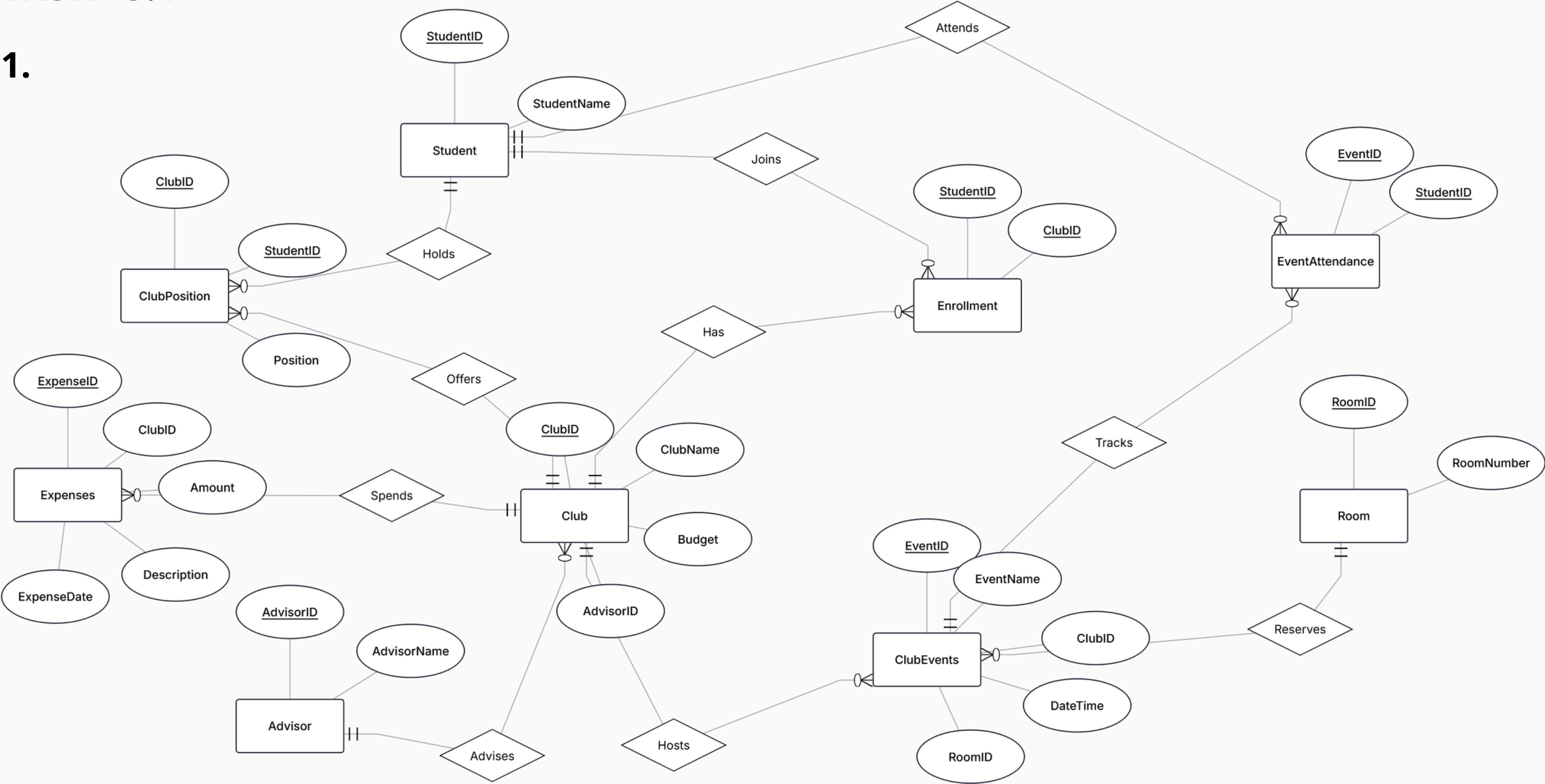
No data is lost. This decomposition is lossless. You can reconstruct the original CourseSchedule table by joining all six tables back together.

• Dependency Preservation (Rule Loss):

Yes, a functional dependency is lost. BCNF decompositions do not always preserve all functional dependencies. In this specific case, the FD CourseID, TimeSlot, InstructorID $\rightarrow$ Room is lost.

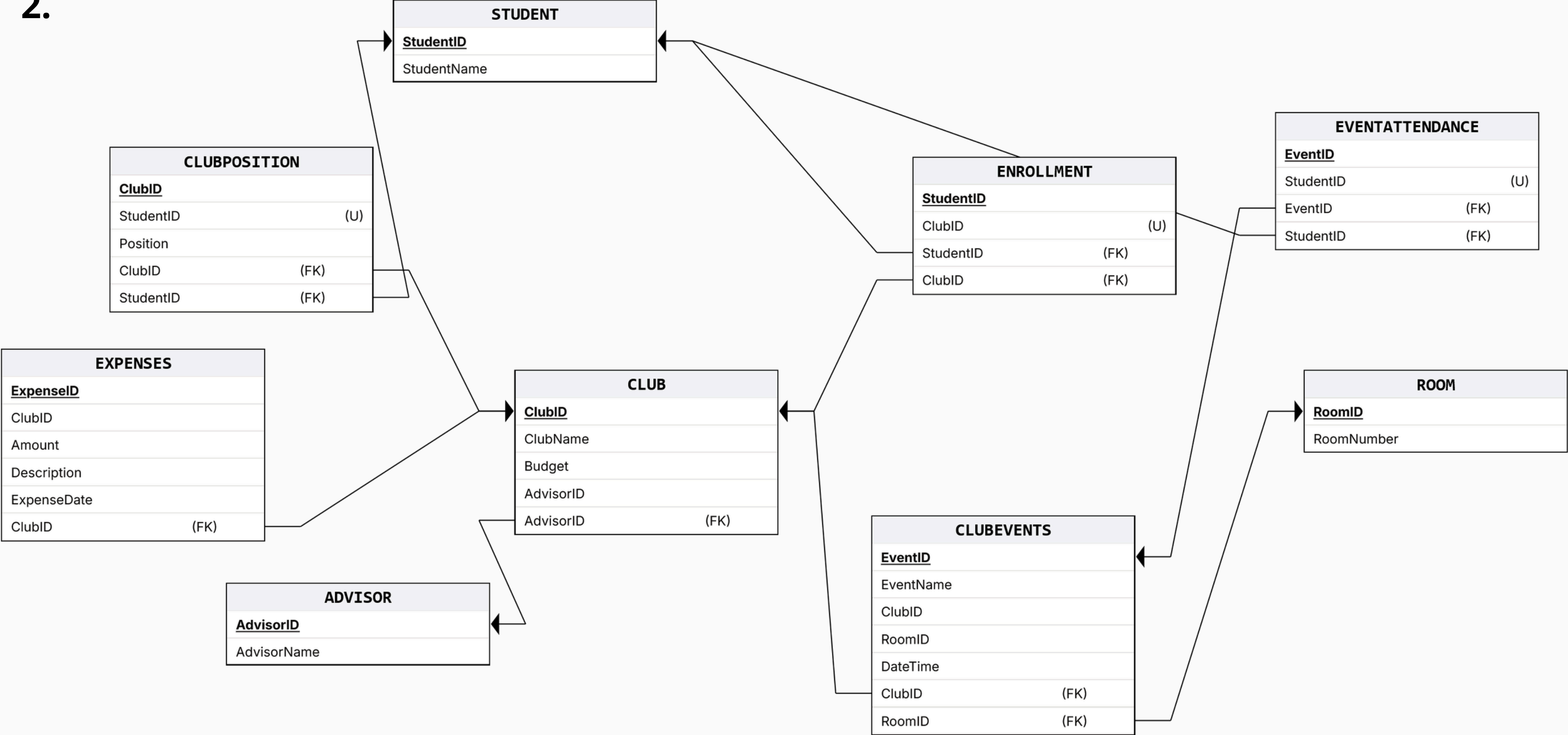
TASK - 5.1

1.



TASK - 5.1

2.



TASK - 5.1

3. Design Decision: Tracking Club Expenses.

Option A: A single Expenses (or TotalSpent) attribute inside the Club table.

Option B: A separate Expenses table that records individual transactions

Explanation: While simple column could work, it would lack the information about those expenses. What expenses were made, when they were made and etc. A table with those expenses is much more detailed and useful for that matter.

Design Decision: Modeling Room Reservations.

Option A: Store a simple RoomNumber text attribute directly inside the ClubEvents table.

Option B: Create a separate Room entity with RoomID and RoomNumber, and link it to ClubEvents using a Foreign Key

Explanation: Addition of a RoomNumber would have been much easier to implement but if there would be multiple rooms with the same number or for example, the row's number of 125 would be changed to 214, then it would make it hard to understand if it is the exact same room which had changed it's number or was the room transferred to another place.

TASK - 5.1

4.
 1. Find all students who hold the 'President' or 'Treasurer' role in any club.
 2. Show what expenses Football club has done in March.
 3. List all rooms that have been reserved for a club event in the past month.